

Fleet Manager Mimarisi ve Algoritmalar

Pickup-dropoff tabanlı çok robotlu filo yönetimi

ROS 2 / Nav2 / Gazebo / FastAPI / SQLite

Bu belge, “Nav2 yığını nedir” konuşmasındaki son revizyonları uygulanabilir bir teknik tasarıma dönüştürür. Görev atama, enerji kontrolü, şarj, hata yönetimi ve yeniden başlatma aynı olay tabanlı yaşam döngüsü içinde tanımlanır.

Tasarımın omurgası

Nav2 robotun hedefe ulaşmasını sağlar. **Fleet Manager** hangi robotun hangi işi ne zaman yapacağını belirler. **battery_simulator** enerji tüketimini ve fiziksel şarj koşullarını modeller. **Domain** iş kurallarını dış teknolojilerden bağımsız tutar.

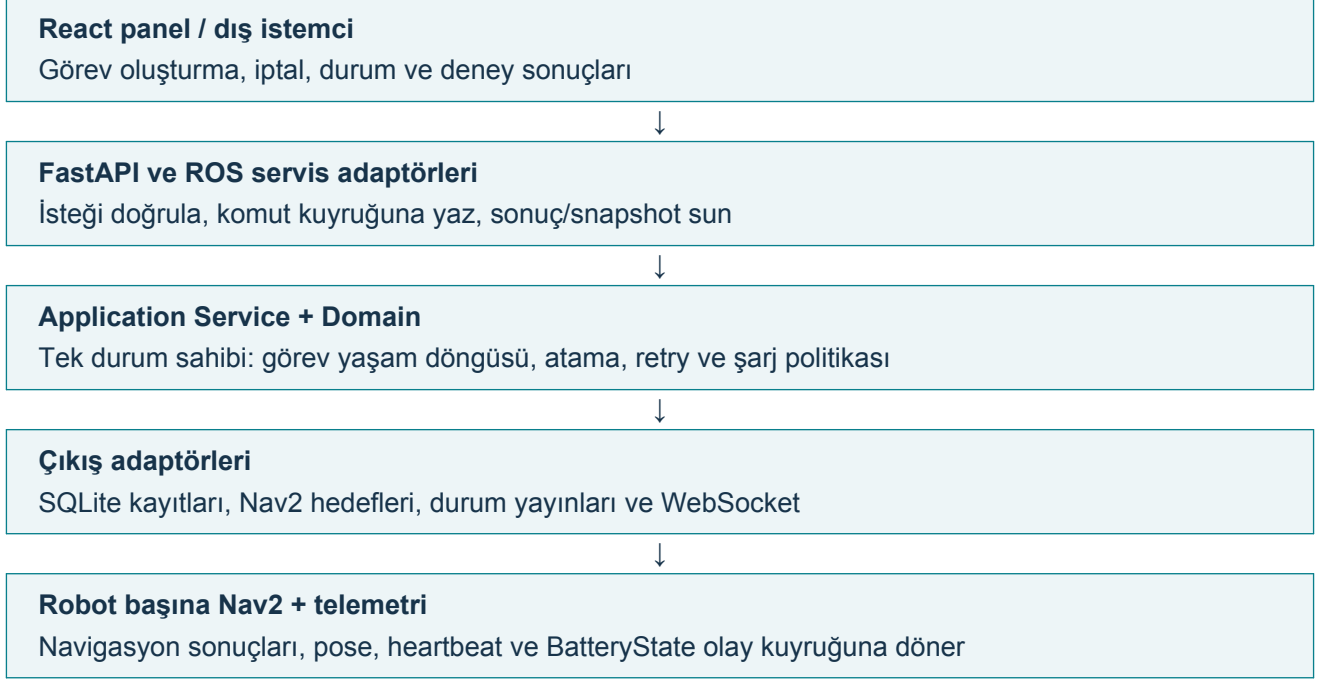
Bölüm	İçerik
02-04	Bileşenler, veri modeli ve durum makineleri
05-07	Event sözleşmeleri, enerji filtresi ve atama stratejileri
08-10	Batarya, hata politikaları ve restart recovery
11-13	Kuyruklar, paket yapısı, arayüzler ve doğrulama

Sürüm: 2.0 | **Tarih:** 8 Ekim 2026

Kapsam: Simülasyon odaklı MVP ve kontrollü genişleme. Eşikler ve katsayılar örnektir; deneylerle kalibre edilir.

Son konuşmada belirtilmeyen ayrıntılar bu belgede “öneri” olarak tamamlanmıştır. Önceki PDF dosyası erişilebilir olmadığı için bu sürüm, konuşmanın tam metnindeki güncel kararlardan yeniden hazırlanmıştır.

Bileşenler ve sorumluluk sınırları



Çekirdek alt bileşenler

Robot Registry: kimlik, son telemetri, kullanılabilirlik ve aktif görev bağını tutar. **Task Queue:** öncelik ve oluşturulma zamanına göre bekleyen işleri sıralar. **Assignment Engine:** uygunluk filtresini ve seçili stratejiyi uygular. **Charging Policy:** iç şarj operasyonlarını yönetir.

Domain içinde rcplpy, ROS mesajı, Nav2, Gazebo veya FastAPI import edilmez. Adaptörler dış veriyi sade domain nesnelere dönüştürür. FastAPI, aynı process içinde ROS servisi üzerinden dolaşmadan ortak application katmanına komut gönderir.

Bu MVP merkezi görev koordinasyonudur. Koridor rezervasyonu, çok robotlu trafik planlama ve çarpışmasız ortak rota planlama ayrıca tasarlanmalıdır; atama stratejisi tek başına bunları sağlamaz.

Veri modeli ve tutarlılık kuralları

Nesne	Önerilen temel alanlar
Task	id, pickup_pose, dropoff_pose, priority, state, assigned_robot_id, payload_state, stage_retry_count, requeue_count, created_at, started_at, completed_at, version
Robot	id, state, pose, battery_soc, usable_capacity_wh, active_task_id, last_seen, telemetry_at, nav_goal_id, charge_phase, error_reason
Event	event_id, type, robot_id, task_id, goal_id, attempt_id, manager_epoch, observed_at, payload
Command	command_id, request_id, type, payload, submitted_at
ChargeOperation	robot_id, station_id, phase, goal_id, deadline, retry_count

Değişmez kurallar

1. Bir robot aynı anda en fazla bir aktif teslimat taşır; bir görev en fazla bir robota bağlıdır.
2. Atama ve robot-görev bağı aynı veritabanı transaction'ında kaydedilir.
3. Teslimatın aşaması için ana gerçek Task.state'tir. Robot.state bunun operasyonel yansımasıdır; OFFLINE, ERROR ve şarj durumu ayrıca robotun kullanılabilirliğini belirler.
4. COMPLETED, FAILED ve CANCELLED terminaldir; tekrar çalıştırma yeni görev veya açık recovery işlemi gerektirir.

Yükün durumu ayrı tutulur

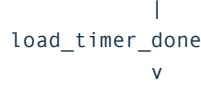
payload_state = NONE / ONBOARD / UNKNOWN. load_timer_done, simülasyon sözleşmesine göre ONBOARD yapar; unload_timer_done NONE yapar. Fiziksel sistemde timer yerine sensör veya işlem onayı kullanılır. LOADING sırasında çökme gibi belirsizliklerde UNKNOWN kaydedilir.

Görev FAILED olsa bile robotta yük kalabilir. active_task_id temizlense dahi payload ve incident bağı korunur; robot operatör çözümü olmadan IDLE olmaz. Pose için ortak map frame'i ve zaman bilgisi zorunludur.

Robot ve task durum makineleri

DELIVERY TASK

QUEUED -> ASSIGNED -> TO_PICKUP -> LOADING



COMPLETED <- UNLOADING <- TO_DROPOFF

Terminal alternatifler: FAILED / CANCELLED

Pickup öncesi retry limiti: QUEUED'a dönüş

Pickup sonrası retry limiti: FAILED

Task aşaması	Robot yansıması	Geçiş koşulu
QUEUED	Bağlı robot yok	Uygun robot beklenir
ASSIGNED	ASSIGNED	Atama kalıcılaştırılmıştır
TO_PICKUP	NAVIGATING_TO_PICKUP	Pickup goal kabul edilmiştir
LOADING	LOADING	Pickup sonucu başarılıdır
TO_DROPOFF	NAVIGATING_TO_DROPOFF	Yükleme bitti; hedef gönderiliyor/aktif
UNLOADING	UNLOADING	Dropoff sonucu başarılıdır
COMPLETED	IDLE veya CHARGING	Boşaltma onayı ve şarj politikası

TO_DROPOFF aşamasında nav_phase = SENDING / ACTIVE / RETRY_WAIT kullanılabilir. Böylece goal gönderimi ile kabulü arasındaki kısa süre ayrı task state üretmeden izlenir. Goal reddedilirse aynı aşamanın hata politikası işletilir.

Robotun bağımsız durumları

OFFLINE: başlangıç veya heartbeat kaybı. **ERROR:** iletişim gelse bile çözülmemiş hata/yük belirsizliği.

CHARGING: şarj operasyonu; charge_phase = TO_STATION / DOCKING / CHARGING / RELEASING.

Gerçek şarj, BatteryState tarafından doğrulanır.

OFFLINE → IDLE yalnızca telemetri tazeyse, navigasyon uzlaştırılmışsa ve yük/hata engeli yoksa gerçekleşir. Tek heartbeat otomatik olarak robotu göreve uygun hale getirmez.

Event ve geçiş tablosu

Event / kaynak	Koşul ve task etkisi	Robot / yan etki
task_submitted / API	Geçerli ve yeni request_id → QUEUED	DB kaydı; atama tetiklenir
robot_selected / engine	QUEUED → ASSIGNED	IDLE → ASSIGNED; pickup gönder
goal_accepted(pickup) / Nav2	ASSIGNED → TO_PICKUP	NAVIGATING_TO_PICKUP
goal_succeeded(pickup) / Nav2	TO_PICKUP → LOADING	LOADING; bloklamayan timer
load_timer_done / timer	LOADING → TO_DROPOFF; ONBOARD	Dropoff goal gönder
goal_accepted(dropoff) / Nav2	TO_DROPOFF kalır; nav ACTIVE	NAVIGATING_TO_DROPOFF
goal_succeeded(dropoff) / Nav2	TO_DROPOFF → UNLOADING	UNLOADING; timer başlat
unload_timer_done / timer	UNLOADING → COMPLETED; NONE	IDLE veya iç şarj operasyonu
goal_rejected/aborted/timeout	Aşama, yük ve denemeye göre retry	Eski goal kapatılmadan yenisi yok
battery_low / policy	Aktif görevde charge_pending=true	IDLE ise istasyona yönlendir
battery_critical / policy	Aktif görev güvenli durdurma/FAILED	ERROR; yük bağı korunur
battery_recovered / policy	SoC >= release ve şarj doğrulandı	Undock sonrası IDLE
heartbeat_lost / registry	Yüksüz: kontrollü requeue; yüklü/belirsiz: FAILED	OFFLINE; goal/ownership karantinası
task_cancel_requested / API	Yük öncesi onaylı iptal → CANCELLED	Goal durması doğrulanır; sonra IDLE
charge_goal_succeeded / Nav2	Teslimat task'ına etkisi yok	DOCKING; contact/şarj bekle

BatteryState sürekli telemetri yayınlar; battery_low / critical / recovered olaylarını eşik geçişlerinden Fleet Manager türetir. Aynı goal_id, attempt_id ve epoch ile eşleşmeyen gecikmiş sonuçlar uygulanmaz.

Yük sonrası iptal isteği MVP'de reddedilir veya operatör çözümüne çevrilir. Robot yükü taşıırken CANCELLED → IDLE geçişi yapılmaz.

Enerji-yeterlilik tabanlı atama

Birimi tutarlı enerji modeli

```

E_available = SoC * C_usable_wh
E_delivery  = k_empty*d(robot,pickup)
              + k_loaded*d(pickup,dropoff)
              + P_aux*(t_drive+t_load+t_unload)/3600
E_return    = k_empty*d(dropoff,charger)
              + P_aux*t_return/3600
E_required  = (1 + uncertainty)*(E_delivery + E_return)

```

Uygunluk: $E_{available} - E_{required} > E_{reserve}$

SoC 0-1 aralığındadır; C_usable_wh kullanılabilir kapasitedir. k katsayıları Wh/m, P_aux W, mesafeler metre ve süreler saniyedir. k yalnız sürüş enerjisini kapsar; yardımcı güç aynı tüketimi iki kez saymamalıdır. E_reserve güvenlik rezervidir; uncertainty model belirsizliğini kapsar.

Mesafe, robot → pickup → dropoff → uygun şarj istasyonu için erişilebilir yol uzunluğudur. Öklid uzaklığı hızlı ön eleme için kullanılabilir; engelli haritada kesin enerji kabulü için yeterli değildir. Geçerli yol bulunamaması veya bayat telemetri robotu uygunsuz yapar.

Sayısal örnek - yalnızca açıklama amaçlı

Kalem	Değer
SoC × kullanılabilir kapasite	0,22 × 200 Wh = 44 Wh
Teslimat + şarja dönüş tahmini	26 + 10 = 36 Wh
%15 belirsizlik dahil ihtiyaç	1,15 × 36 = 41,4 Wh
Kalan enerji / rezerv	44 - 41,4 = 2,6 Wh; rezerv 10 Wh
Karar	Uygun değil: 2,6 > 10 koşulu sağlanmaz

Bu örnekte robot %22 seviyesinde olsa da göreve atanmaz. Ayrıca soft_low altındaki robotlar yeni işe alınmaz. Eşik, enerji denkleminde ek bir işletim filtresidir; denklemin yerine geçmez.

Şarj istasyonu seçimi

Erişilebilir, kapasitesi uygun istasyonlar arasından dönüş maliyeti en düşük olan seçilir. Dolu istasyon için bekleme enerjisi eklenir. MVP'de robot başına istasyon veya basit rezervasyon kullanılır; sınırsız erişim varsayımı açıkça deney koşuluna yazılır.

Atama algoritmaları ve stratejiler

```

assign(task):
    candidates = []
    for robot in registry:
        if not idle_online_fresh_empty(robot): continue
        if robot.soc <= soft_low: continue
        estimate = estimate_route_energy(robot, task)
        if not estimate.reachable: continue
        if robot.energy - estimate.required <= reserve: continue
        candidates.append((robot, estimate))
    selected = strategy.select(task, candidates)
    if selected is None: return KEEP_QUEUED
    transaction:
        recheck_availability_and_version(selected, task)
        bind_robot_and_task(); save_assignment_and_outbox()
    dispatch_pickup_after_commit()

```

Strateji	Seçim kuralı	Kullanım / sınır
Random	Uygun adaylar arasından uniform seçim	Deney tabanı; seed kaydedilir
Nearest	$\text{argmin } d(\text{robot}, \text{pickup})$	Yol uzunluğu tercih edilir; toplam maliyeti görmez
CostBased	argmin normalize edilmiş ağırlıklı maliyet	Enerji, süre ve yakınlığı dengeler

```

J(r, t) = wd * d_pickup/D_ref
        + we * E_required/E_ref
        + wt * ETA_delivery/T_ref
        + wb * reserve/E_after
E_after = E_available - E_required
wd + we + wt + wb = 1; weights >= 0

```

Referans ölçekler sabit ve pozitifdir; tüm stratejiler aynı uygun aday filtresini kullanır. $\text{reserve}/E_{\text{after}}$, enerji marjı dar robotu cezalandırır. Örnek ağırlıklar: $wd=0,35$; $we=0,30$; $wt=0,20$; $wb=0,15$. Bunlar öneridir; deneylerle ayarlanır.

Görev sırası: priority azalan, created_at artan. Uzun bekleyen görevler için aging önerilir. Eşit robot puanında robot_id ile deterministik bağ kırma kullanılır. Yeniden kuyruğa alma sonrasında cooldown ve requeue limiti sonsuz denemeyi engeller.

Arayüz: $\text{select_robot}(\text{task}, \text{eligible_candidates}, \text{context}) \rightarrow \text{robot_id} \mid \text{None}$. Domain stratejileri ROS nesnesi yerine yol/enerji tahminlerini alır. Bu sürüm çevrimiçi greedy atamadır; küresel optimum garantisi vermez.

Batarya simülasyonu ve şarj histerezi

```

/robot_i/odom + map pose + /robot_i/dock_contact
|
battery_simulator
|
/robot_i/battery_state
|
Fleet Manager: enerji/şarj politikası

```

Simülatör, Fleet Manager'dan "şarj oluyor" komutu almaz. İstasyon bölgesinde, hareketsiz ve dock/contact doğrulanmış robot şarj edilir. Odom frame ile map frame farklıysa istasyon kontrolü için dönüşüm uygulanır. Hız ve temas kısa bir doğrulama süresi boyunca kararlı olmalıdır.

```

charging = in_zone(pose) and stationary(v, omega) and stable_dock_contact
if charging:
    E = min(C_usable, E + eta_charge * P_charge * dt / 3600)
else:
    E = max(0, E - k_motion * distance - P_aux * dt / 3600)
publish BatteryState(percentage=E/C_usable,
                    power_supply_status=physical_status)

```

Parametre	Örnek	Politika
soft_low	0,25 (%25)	Yeni iş alma; boşsa şarja git
critical_low	0,08 (%8)	Acil durum; güvenli duruş ve ERROR
charge_release	0,80 (%80)	Şarj oturumunu bitirme hedefi
safety reserve	10 Wh (örnek)	Enerji denklemindeki zorunlu pay

Şarj talebi kilitlenir: 0,25 altına inilince oturum başlar; 0,25 üstüne çıkmak yeterli değildir, 0,80 hedefi beklenir. Contact kaybı, stale batarya veya dock timeout şarjın başarılı sayılmasını engeller.

Aktif teslimatta soft_low, görevi kesmeden charge_pending işaretler. Kritik eşik veya kalan rota için enerji yetersizliği güvenlik politikasını tetikler. Nav2 iptali ve duruş doğrulanır; yüklü robot yeniden pickup kuyruğuna alınmaz.

CHARGING durumunun TO_STATION aşaması fiziksel şarj değildir. İstasyona Nav2 ile ulaşmak contact onayı yerine geçmez. Hedef enerji sonrası undock/istasyondan ayrılma doğrulanınca IDLE yapılır. Zaman hesabı simülasyonda /clock ile tutarlı tutulur.

Hata, retry ve iptal politikaları

Durum	Uygulanacak politika
Pickup öncesi Nav2 hatası	Aynı hedefte sınırlı retry. Limitte robot ERROR, görev QUEUED; bağ kaldırılır. Eski hedef kapanması doğrulanır.
Yükleme sırasında belirsizlik	Payload UNKNOWN; robot ERROR, görev FAILED. Otomatik başka robota atama yapılmaz.
Pickup sonrası navigasyon hatası	Yük ONBOARD korunur. Aynı robot aynı dropoff hedefine retry. Limitte robot ERROR, görev FAILED.
Boşaltma sırasında hata	Yükün bırakıldığı doğrulanmıyorsa UNKNOWN/ONBOARD korunur. FAILED ve operatör çözümü.
Heartbeat kaybı	Robot OFFLINE. Yüklü/belirsiz görev FAILED; yüksüz görev ancak eski yürütme durdurulunca requeue.
Pickup öncesi iptal	Goal cancel sonucu ve duruş doğrulanır; task CANCELLED. Robot uygunluk kontrolünden sonra IDLE.
Pickup sonrası iptal	MVP'de reddet; teslimatı sürdür veya operatörce güvenli boşaltma/recovery.
Kritik batarya	Güvenli duruş isteği, ERROR ve aktif görev FAILED; payload kaydı korunur.

```

on_navigation_failure(event):
    if not matches_current_goal_and_epoch(event): return
    close_or_cancel_old_goal()
    if termination_unconfirmed(): quarantine_robot(); return
    if retries < max_retries:
        schedule_retry(same_robot, same_target, backoff)
    elif payload == NONE and stage_before_loading:
        mark_robot_error(); requeue_with_budget(task)
    else:
        mark_robot_error(); fail_task_keep_payload_link()

```

Örnek retry bütçesi: max_retries=2, yani ilk deneme dahil en fazla 3 girişim/aşama. Yeniden atama sayacı ayrı tutulur; requeue limiti aşılsa görev FAILED olur. Backoff timer ile uygulanır; callback içinde sleep kullanılmaz.

Timeout, robotun gerçekten durduğu anlamına gelmez. Önce cancel/terminal sonuç ve duruş kontrolü gerekir. İletişim yoksa robotun eski komutu sürdürme ihtimali bulunduğundan görev sahipliği karantinaya alınır.

Restart recovery ve kalıcı kayıtlar

Başlangıçta atama kapısı kapalıdır; tüm robotlar OFFLINE kabul edilir. Veritabanındaki görev aşaması, yük durumu ve eski Nav2 yürütmesi uzlaştırılmadan yeni hedef gönderilmez. Bu, konuşmadaki muhafazakâr restart politikasının uygulanabilir genişletmesidir.

DB'deki task	Recovery kararı
QUEUED	Korunur; uygun robot geldikten sonra atanabilir.
ASSIGNED / TO_PICKUP	Payload NONE ve eski goal durduğu doğrulanınca QUEUED. Doğrulanamazsa atama kapısı kapalı/robot karantinada.
LOADING	Yük belirsiz kabul edilir: FAILED; robot ERROR ve kontrol gerekir.
TO_DROPOFF	FAILED; ONBOARD bağı korunur. Operatör aynı robotla dropoff recovery başlatabilir.
UNLOADING	FAILED; yük bırakma durumu doğrulanır. Körlemesine tekrar pickup yapılmaz.
COMPLETED / FAILED / CANCELLED	Terminal kayıt korunur; payload incident ayrıca korunur.

1. **Açılış kilidi** - manager_epoch üret, atamayı durdur, kayıtları yükle



2. **Robot uzlaştırma** - telemetri, pose, batarya, yük ve eski goal bilgisi



3. **Güvenli durdurma** - eski yürütmeyi iptal et/doğrula; belirsiz robotu ayır



4. **Recovery transaction** - aşamaya göre requeue veya FAILED, incident kaydı



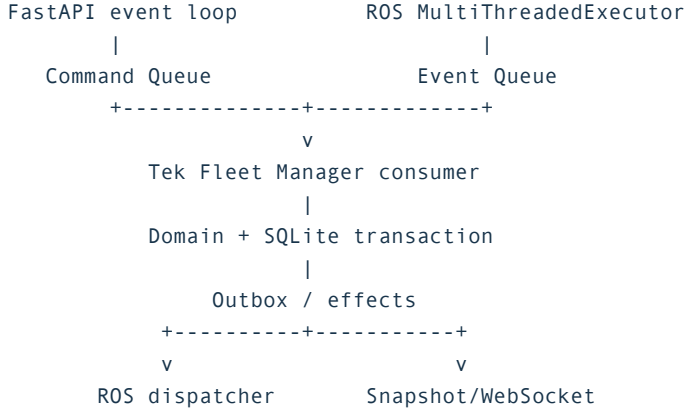
5. **Atamayı aç** - yalnızca uzlaştırılmış robotları aday yap

SQLite önerisi

tasks, robots, task_events, commands, outbox ve incidents tabloları. Durum güncellemesi + event journal + dış etki kaydı tek transaction'da tutulur. DB bağlantısını tek writer kullanır; sürüm alanı yarışları ve eski komutları yakalar.

DB transaction ile ROS action gönderimi atomik değildir. Outbox dış etkinin niyetini saklar; yeniden gönderimde goal/attempt kimliği ve uzlaştırma kullanılır. "Exactly once" varsayılmaz. SQLite tek process MVP için yeterlidir; WAL ve kontrollü transaction süreleri önerilir.

Event/command queue ve threading



FastAPI ayrı thread/event loop'ta, ROS callback'leri executor'da çalışabilir. Hiçbiri paylaşılan Robot/Task nesnesini değiştirmez. Domain durumunun tek sahibi consumer'dır. API okumaları değişmez snapshot üzerinden yapılır; sonuç gerekiyorsa request_id ile future tamamlanır.

```

while running:
    item = next_command_or_event_with_fairness()
    if duplicate_or_stale(item): continue
    effects = application.handle(item)
    commit_state_events_and_outbox()
    publish_immutable_snapshot()
    dispatch_nonblocking(effects)
  
```

Kuyruk ve callback kuralları

Kuyruklar thread-safe ve bounded olur. Pose/batarya gibi yüksek frekanslı telemetri robot başına son değerle birleştirilebilir. Goal sonucu, iptal sonucu ve kritik olaylar kaybedilmez; kapasite dolarsa backpressure veya kalıcı journal uygulanır.

Event ve command kuyruğunda adil tüketim yapılır; kritik olayların gecikmesi sınırlanır. Deadline/timer olayları bloklamadan üretilir. ROS gönderimleri asenkron yürütülür; callback'ler future sonucunu event'e çevirir. Callback group seçimi, eşzamanlılık gereksinimine göre açıkça yapılır.

Mesaj zarfında event_id, task_id, robot_id, goal_id, attempt_id ve manager_epoch bulunur. Tekrarlanan API request_id aynı görevi döndürür. Gecikmiş önceki deneme sonucu yeni göreve uygulanmaz. Kapanışta yeni komut kabulü durdurulur ve kalıcı kayıtlar tamamlanır.

Paket yapısı ve fleet_interfaces

```
fleet_ws/src/
  fleet_interfaces/  msg/ + srv/ (ament_cmake)
  fleet_manager/
    domain/         models, states, events, rules
    application/     services, recovery, charging
    assignment/      base, random, nearest, cost_based
    ros_adapters/    nav2, telemetry, services, publisher
    persistence/     repositories, sqlite, outbox
    api/             routes, schemas, websocket
    config/          policy ve strateji parametreleri
    tests/           domain, integration, recovery
  battery_simulator/ model, dock input, BatteryState
  fleet_sim/         world, URDF, launch, Nav2 params
  frontend/         React + Vite; workspace dışında
```

Standart arayüzleri kullan: sensor_msgs/BatteryState, geometry_msgs/PoseStamped ve nav2_msgs/action/NavigateToPose. Custom mesajlar yalnız filo kavramlarını kapsar. Robot başına /robot_i namespace'i; ortak veya açıkça dönüştürülmüş map frame'i kullanılır. [1, 2]

Arayüz	Önerilen alanlar / sonuç
RobotStatus.msg	Header, robot_id, state, charge_phase, battery_soc, PoseStamped pose, active_task_id, payload_state, error_reason
TaskStatus.msg	task_id, state, pickup/dropoff PoseStamped, assigned_robot_id, stage_retry_count, requeue_count, payload_state, zamanlar, version
SubmitTask.srv	request_id, pickup, dropoff, priority → accepted, task_id, reason
CancelTask.srv	request_id, task_id → accepted, reason; accepted tamamlanmış iptal değildir
SetAssignmentStrategy.srv	strategy_name → accepted, active_strategy, reason
FleetStatus.msg (opsiyonel)	Header, RobotStatus[], TaskStatus[] veya özet sayaçlar

uint8 state kullanılıyorsa enum sabitleri mesajda tanımlanır; sürümler arasında anlamı korunur. BatteryState.percentage 0-1 ölçeğindedir; UI yüzdeyi $\times 100$ gösterir. NaN veya bayat veri uygunluk filtresinde reddedilir. [1]

Önerilen HTTP uçları: POST /tasks, POST /tasks/{id}/cancel, GET /tasks, GET /robots, PUT /assignment-strategy; WebSocket /ws/fleet. Komut kabulü için HTTP 202 ve command_id; gerçek sonuç durum akışından izlenir. API ile ROS servisleri aynı komut modelini kullanır.

Doğrulama, deneyler ve kaynaklar

Uygulama sırası

1. Saf domain modeli, geçiş kuralları ve enerji filtresi.
2. SQLite, event/command kuyruğu, outbox ve restart recovery.
3. Tek robotla Nav2 pickup-dropoff ve timer akışı.
4. Batarya, contact tabanlı şarj ve kritik eşik.
5. Çok robotlu namespace yapısı, üç strateji ve web paneli.

Kabul senaryoları

Senaryo	Beklenen sonuç
%22 batarya; enerji rezervi yetersiz	Atama reddedilir; görev QUEUED kalır.
Pickup öncesi 3 başarısız girişim	Robot ERROR; bütçe uygunsa görev requeue.
Pickup sonrası retry limiti	Aynı robot retry; limitte FAILED, yük bağı korunur.
Şarj bölgesinde temas olmadan duruş	Fiziksel şarj raporlanmaz.
Şarj oturumunda %26 → %79 → %80	Oturum %80 hedefinden önce bırakılmaz.
LOADING/TO_DROPOFF sırasında restart	Görev FAILED; robot otomatik IDLE olmaz.
Eski goal sonucu ve tekrar API isteği	Yanlış geçiş veya çift görev oluşturulmaz.
Heartbeat kaybı; eski goal belirsiz	Robot atama dışı; körlemesine devir yapılmaz.

Strateji karşılaştırmasında aynı harita, görev dizisi, başlangıç pose/SoC, enerji modeli ve istasyon kapasitesi kullanılır. Random için birden fazla seed denenir. Tamamlanma oranı, bekleme süresi, teslimat süresi, yol uzunluğu, Wh tüketimi, retry ve şarj sayısı raporlanır. Nearest ve CostBased karşılaştırması yalnız aynı uygun aday kümesiyle anlamlıdır.

Kaynak ve sürüm notları

Temel tasarım kaynağı: “Nav2 yığını nedir” konuşmasının son mimari revizyonu. Bu belge içindeki retry bütçesi, maliyet fonksiyonu, kuyruk sınırları, outbox ve payload alanları uygulama önerileridir.

1. ROS 2 sensor_msgs/BatteryState mesaj tanımı
2. Nav2 NavigateToPose action belgesi

Standart arayüzler için Rolling kaynakları kontrol edilmiştir. Uygulamada seçilen ROS 2 dağıtımının kurulu mesaj/action tanımları esas alınmalıdır; özellikle Nav2 sonuç alanları sürüme göre değişebilir. Belge herhangi bir dağıtımı zorunlu kılmaz.